VUIBERT PRÉPAS

A. Caignot • M. Dérumaux • J. Labasque • L. Moisan

INFORMATIQUE POUR TOUS

Classes préparatoires scientifiques 1re et 2e années

Tout le programme avec PYTHON et SCILAB

Tout-en-un

- Tout le cours
- Fiches de synthèse
- Conseils méthodologiques
- Vrai/faux
- Exercices d'application
- Problèmes de synthèse
- Sujets de concours
- Tous les corrigés détaillés

Vuibert

Avant-propos

Cet ouvrage vous propose, en un seul volume, toutes les clés nécessaires pour réussir votre première et votre deuxième année d'informatique en classes préparatoires scientifiques.

Cours complet

Rigoureusement conforme aux nouveaux programmes, il contient tous les outils pour acquérir les connaissances et les savoir-faire indispensables.

Fiches de synthèse

Pour une révision efficace avant les épreuves, l'essentiel du cours est présenté de manière synthétique sous forme de fiches de révision.

Vrai/faux

Première étape vers l'entraînement, des Vrai/faux vous permettent de tester rapidement la compréhension du cours.

Exercices d'application et problèmes de synthèse

Application directe du cours, ces nombreux exercices et problèmes sont assortis d'un corrigé détaillé. Chacun a un niveau de difficulté clairement identifié : ●○○, ●●○ ou ●●●.

Sujets de concours

Pour se mettre en situation d'épreuve, une sélection d'exercices extraits de sujets de concours vous est proposée. Tous ces exercices sont intégralement corrigés.

Retrouvez sur le site www.vuibert.fr, à la page du livre,
des ressources complémentaires à télécharger

Table des matières

Préface . . . . . **VII**

Chapitre 1. **Architecture des ordinateurs** **1**

1. Les composants et leurs connexions **1** – 2. L'architecture logicielle **5** – 3. Le codage des données en mémoire **7**

Chapitre 2. **Programmation et algorithmique** **19**

1. L'installation et l'utilisation de Python et Scilab **19** – 2. Les structures de données **25** – 3. Les instructions de contrôle **39** – 4. Les fonctions **43** – 5. Le débogage d'un programme **50** – 6. La lecture et l'écriture de fichiers **54** – 7. Les tracés graphiques **56** – 8. La terminaison, la correction et la complexité **56** – **Synthèse 64** – **Exercices 66** – **Corrigés 86**

Chapitre 3. **Simulation numérique** **111**

1. ✷ La vectorisation des calculs **111** – 2. ♣ La résolution des équations non linéaires **117** – 3. ♠ L'intégration et la dérivation de signaux numériques **129** – 4. ♥ L'intégration des équations différentielles **137** – 5. ♦ La résolution d'un système linéaire par la méthode de Gauss **146** – **Synthèse 152** – **Exercices 154** – **Corrigés 182**

Chapitre 4. **Initiation aux bases de données** **219**

1. Les motivations **219** – 2. L'architecture matérielle et logicielle des bases de données **220** – 3. La modélisation d'une base de données **222** – 4. L'utilisation d'un logiciel pour manipuler des bases de données **225** – 5. L'algèbre relationnelle **227** – **Synthèse 237** – **Exercices 239** – **Corrigés 244**

Chapitre 5. **Algorithmique : approfondissement** **247**

1. Les piles **247** – 2. Les fonctions récursives **252** – 3. Les algorithmes de tri **259** – 4. La programmation orientée objet **267** – **Synthèse 273** – **Exercices 275** – **Corrigés 283**

Chapitre 6. **Problèmes de synthèse** **299**

1. Les checksums et la correction des erreurs **299** – 2. La traduction automatique avec un arbre **302** – 3. La triangulation d'un polygone **307** – 4. La propagation de la chaleur dans une ailette **313** – 5. Autour des nombres premiers **318** – 6. Transformation homographique d'une image numérique **325** – 7. La compression RLE des images clipart **332** – 8. La compression LZW **337** – 9. La compression de Huffman d'un texte **341** – 10. Une balade dans le métro parisien **346**

Préface

Depuis leur création à la fin du XIXe siècle (par Henri Vuibert, alors plus jeune agrégé de mathématiques de France) les Éditions Vuibert proposent des manuels scientifiques rédigés par les meilleurs auteurs, tous professeurs passionnés par leur discipline et leur enseignement.

Ce fut donc avec un très grand plaisir que je fus contacté pour diriger une nouvelle collection d'ouvrages scientifiques destinés aux étudiants préparationnaires, en adéquation avec les nouveaux programmes de la rentrée 2013.

Nous avons réuni pour cette tâche difficile des auteurs de grand talent, aussi bien pour leur qualification disciplinaire que pour leur désir de communiquer leur savoir à un public de plus en plus hétérogène.

Entre 1980 et 2010, le nombre d'étudiants de CPGE scientifique a plus que doublé, de nouvelles sections ont vu le jour, des classes ont ouvert dans un grand nombre de villes ; pendant cette période, la formation initiale scientifique des élèves à la sortie de l'enseignement secondaire a beaucoup évolué, en même temps que s'érodait le nombre d'heures alloué aux disciplines scientifiques.

L'écart s'est donc creusé entre la terminale et les classes préparatoires aux grandes écoles. Il revient alors aux manuels, comme aux professeurs, de faire preuve de qualités pédagogiques exceptionnelles, sans jamais sacrifier la rigueur indispensable qui est une des forces de l'enseignement supérieur « à la française ». C'est dans ce but que les livres de la collection Vuibert Prépas ont été pensés et rédigés. Ils sont destinés au plus grand nombre et visent à amener ce plus grand nombre au niveau de l'excellence.

Le rôle d'un manuel de classe préparatoire n'est pas évident. Les étudiants disposent déjà de leurs notes de cours, et parfois de polycopiés, provenant d'enseignants fort compétents. Mais chacun sait qu'on observe mieux une statue et qu'on en apprécie mieux la beauté en la regardant sous différents angles ; il en est de même des disciplines scientifiques dans lesquelles une diversité d'approches ne peut que faciliter la compréhension et l'assimilation de notions a priori abstraites et difficiles. En ce sens, les ouvrages de la collection « Vuibert Prépas » constituent une aide conséquente pour les élèves de CPGE scientifiques.

À lire ces ouvrages, que ce soit dans les disciplines qui sont les miennes, Mathématiques et Informatique ou dans des disciplines qui me sont moins familières comme la Physique, la Chimie ou les Sciences de l'Ingénieur, je ne peux être qu'admiratif devant le talent des auteurs de toutes origines qui, dans des délais très courts, ont eu à cœur de faire passer leur amour pour la science et pour son enseignement.

Je suis certain que le public préparationnaire partagera mon enthousiasme pour cette collection qui marque le retour des éditions Vuibert au service de ces filières.

Denis Monasse

Chapitre 4

Initiation aux bases de données

Certains exercices et applications du cours utilisent des fichiers disponibles dans un dossier ressource en libre téléchargement à la page du livre sur le site `http://www.vuibert.fr/`.

1. Les motivations

La gestion des données numériques est une des problématiques les plus importantes de ces dernières années. En effet, la très grande majorité des entreprises ne travaille plus qu'avec des données numériques, de telle sorte que très peu d'informations sont encore stockées sur du papier.

Les données à conserver, traiter, trier sont souvent hétérogènes (nombres, textes, fichiers...) et en très grand volume. Ainsi, la structure la plus simple pour classer des données est le tableau à deux dimensions. Or, si cette méthode est simple, elle présente néanmoins un certain nombre de limitations...

Exemple

L'exemple que nous allons utiliser dans ce cours est une base de données permettant de gérer une bibliothèque : cette base doit permettre de connaître la liste des livres de la bibliothèque et de savoir qui a emprunté chaque livre, donc de gérer également la liste des adhérents de la bibliothèque. La structure retenue se voudra volontairement élémentaire pour que les illustrations restent claires.

Le tableau permettant de gérer la bibliothèque possède la structure suivante :

titre	auteur	editeur	nb_pages	annee	nom_emprunteur	prenom	email_emprunteur	date_retour
Notre-Dame de Paris	Victor Hugo	Gosselin	636	1831	DUPONT	Jean	jean.dupont@gmail.com	05/06/2014
Les misérables	Victor Hugo	Lacroix	1662	1862				
Voyage au centre de la Terre	Jules Verne	Hetzel	372	1864	DUPONT	Jean	jean.dupont@gmail.com	10/06/2014
Le tour du monde en 80 jours	Jules Verne	Hetzel	223	1872	MARTIN	Paul	paul.martin@gmail.com	10/07/2014

Table 4.1. Table complète de la bibliothèque.

Cet exemple élémentaire permet de mettre en évidence un certain nombre de problèmes :

- les informations sur l'emprunteur sont redondantes ;
- en cas de modification des informations d'un emprunteur, il faut le faire sur toutes les lignes ;
- en cas de suppression d'un livre ou si l'adhérent n'emprunte aucun livre, toutes ses données sont supprimées.

Remarque

Le nombre d'informations concernant l'emprunteur a été volontairement minimisé, mais elles pourraient être largement complétées. De même, les informations relatives aux auteurs (année de naissance, de décès…), aux éditeurs (adresse…) pourraient être bien plus nombreuses, ce qui augmenterait considérablement le nombre de données redondantes.

2. L'architecture matérielle et logicielle des bases de données

Une application ou un logiciel est destiné à manipuler des données et à les traiter pour produire un résultat sous forme de données graphiques (texte, courbe, fenêtre, bouton…), sous forme de fichiers texte, etc.

En fonction du type d'application et de la nature des données à manipuler, différents types d'architectures matérielles et logicielles peuvent être utilisés. On se limite ici uniquement au cas où l'application a besoin de manipuler des données stockées dans une base de données.

2.1. Architecture à 1 niveau

Il s'agit d'une base de données fonctionnant en local sur l'ordinateur de l'utilisateur.

Ce cas limite extrêmement élémentaire concerne des applications dont les données n'ont pas à être partagées avec d'autres utilisateurs. Un grand nombre de logiciels, comme les logiciels de messagerie, utilisent des bases de données pour stocker et manipuler les données (carnet d'adresse, messages…). Ce cas limite sera celui utilisé pour illustrer le cours avec des bases de données SQLite.

Dès lors que les données doivent être partagées entre différents utilisateurs, il faut passer sur une architecture client-serveur.

2.2. Architecture à 2 niveaux

Une architecture à 2 niveaux ou 2 tiers est une architecture composée de clients et d'un serveur (voir figure 4.1).

En fonction du type de données à utiliser, du type d'application, la répartition des rôles entre le client et le serveur peut être différente :

- l'application nécessite de petits volumes de données avec beaucoup de traitements : pour ne pas surcharger le serveur avec le traitement des données, celui-ci se contentera d'envoyer les données au client qui les traitera en local ;
- l'application nécessite de manipuler de très gros volumes de données : pour ne pas perdre de temps dans la transmission de la totalité des données, le serveur traitera les données et enverra uniquement le résultat au client.

Ce type d'architecture reste viable tant que le nombre de clients qui accèdent à la base de données reste inférieur à une centaine de connexions. Quand le nombre de connexions devient très grand sur des volumes de données importants, il faut passer à des architectures à 3 niveaux.

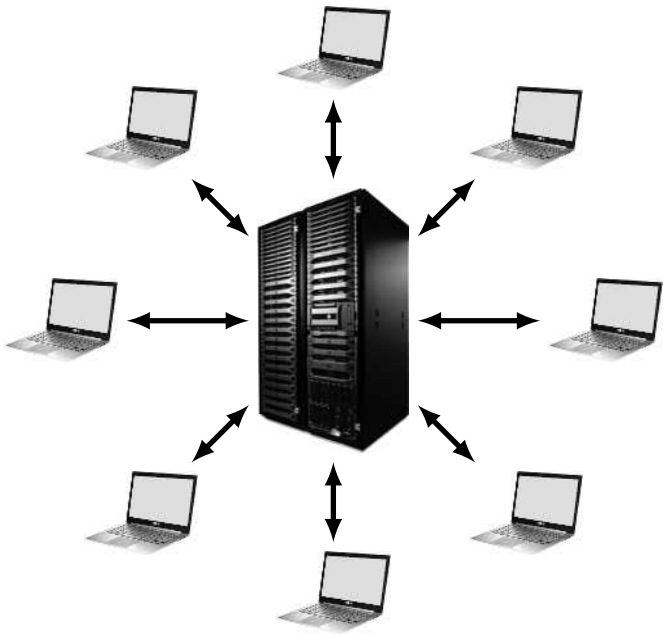


Figure 4.1. Architecture client-serveur à 2 niveaux.

2.3. Architecture à 3 niveaux

Une architecture à 3 niveaux ou 3 tiers est l'architecture classique rencontrée sur les applications internet (voir figure 4.2). Elle est composée :

- d'un client. Par exemple, le navigateur internet qui demande à afficher une page. Il recevra les informations du serveur et se chargera de la mise en page localement ;
- d'un serveur, qui va traiter la demande du client. Par exemple, le serveur va devoir assembler les contenus à afficher sur le navigateur. Mais, parmi ces contenus, certains sont dans une base de données, le serveur envoie alors une requête à un serveur de base de données sous format SQL ;
- d'un serveur de bases de données. Celui-ci va traiter les requêtes SQL et envoyer les données nécessaires au serveur.

L'objectif est de répartir un maximum la charge entre l'accès aux données (géré par le serveur de bases de données) et le traitement des données (géré par le serveur). Le client quant à lui se contente la plupart du temps d'afficher les contenus.

2.4. Architecture à N niveaux

Dans les services de données extrêmement complexes, le nombre de niveaux augmente. Chaque niveau (sauf le client) sera composé d'un serveur qui sera dédié à une tâche particulière, l'objectif étant de distribuer les différentes opérations pour paralléliser le traitement et améliorer la rapidité d'exécution de l'ensemble de l'application.

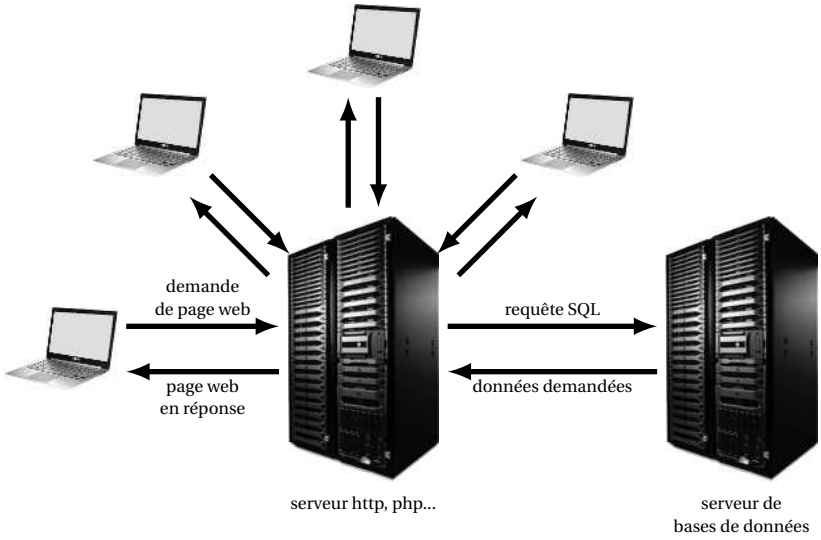


Figure 4.2. Architecture client-serveurs à trois niveaux.

3. La modélisation d'une base de données

Définition 4.1. Base de données

Une base de données est un ensemble structuré d'informations qui permet de répondre à un besoin spécifique.

Pour pouvoir être consultée, modifiée, créée, il est nécessaire d'avoir un logiciel de gestion des bases de données qui va permettre de faire le lien entre une application cliente et les données de la base.

Les logiciels de gestion de bases classiques sont basés sur le langage de requête structurée SQL (*Structure Query Language*), il en existe des payants, comme Microsoft SQL Server, par exemple, ainsi que des gratuits tels que MySQL, PostgreSQL, SQLite...

3.1. Modélisation par entité/association

Les éléments du monde réel que nous pouvons percevoir ne peuvent pas tous être représentés. Il convient de faire un choix pour délimiter la représentation de ces éléments en fonction des objectifs visés par l'application. Chaque élément du monde réel est représenté par une entité. Chaque entité possède des attributs : la liste des paramètres qui caractérisent l'entité. Il convient ensuite de définir les associations entre les différentes entités, c'est-à-dire les relations entre les différentes entités. On parle de modélisation par entité/association.

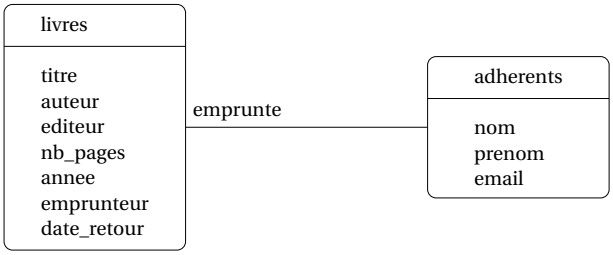
Exemple

Dans le cas de la bibliothèque, pour gérer uniquement la liste des livres et la liste des adhérents, il faut définir deux entités :

- les livres, entité qui contient les attributs suivant :
  - titre : texte,
  - auteur : texte,
  - editeur : texte,
  - nb_pages : nombre,
  - annee : date format AAAA,
  - emprunteur : lien vers un adhérent à définir,
  - date_retour : date format JJ/MM/AAAA ;
- les adhérents, entité qui contient :
  - nom : texte,
  - prenom : texte,
  - email : texte.

Il existe différents outils graphiques permettant de représenter une modélisation par entité/association, nous illustrerons ici uniquement la syntaxe UML. Elle possède deux éléments graphiques de base :

- un ensemble d'entité, qui est représenté par un rectangle à bord arrondi dans lequel se trouve le nom de l'ensemble des entités et le nom de leurs attributs ;
- une association, qui est représentée par un trait entre deux entités.



3.2. Vocabulaire

Le modèle utilisé pour représenter une base de données est relativement simple : c'est celui de plusieurs tableaux ou tables à deux dimensions. Par convention, les données des différents éléments sont en ligne et les informations sont en colonne. Voici un exemple simple que nous utiliserons par la suite : les livres d'une bibliothèque ainsi que les emprunteurs, présentés les deux tables 4.2 et 4.3.

Quelques définitions :

- les titres de colonnes sont les attributs notés $A_1, A_2, \ldots, A_p$;
- l'ensemble des valeurs possibles d'un attribut A est son domaine noté $Dom(A)$;
- chaque ligne est un *n-uplet* et un élément de $Dom(A_1) \times Dom(A_2) \times \cdots \times Dom(A_p)$;
- un schéma relationnel ou une relation R est défini :
  - par un ensemble fini et fixé d'attributs et de leur domaine noté S,
  - et par l'ensemble des *n-uplets* appelée extension de la relation.

id	titre	auteur	editeur	nb_pages	annee	emprunteur	date_retour
1	Notre-Dame de Paris	Victor Hugo	Gosselin	636	1831	1	05/06/2014
2	Les misérables	Victor Hugo	Lacroix	1662	1862		
3	Voyage au centre de la Terre	Jules Verne	Hetzel	372	1864	1	10/06/2014
4	Le tour du monde en 80 jours	Jules Verne	Hetzel	223	1872	2	10/07/2014

Table 4.2. Table des livres.

id	nom	prenom	email
1	DUPONT	Jean	jean.dupont@gmail.com
2	MARTIN	Paul	paul.martin@gmail.com

Table 4.3. Table des adhérents.

Par définition, les attributs forment un ensemble, ce qui implique qu'il n'y a pas d'attribut en double et que l'ordre des attributs n'est pas fixé : on ne parle pas du premier attribut mais de l'attribut titre (dans notre exemple).

Il est possible de restreindre une ligne à un sous-ensemble d'attributs, soit r un *n-uplet*, alors $r(A_2, A_5, A_3)$ est la restriction de r aux attributs A_2, A_5 et A_3. De manière générale, si S est une partie de $\{A_1, A_2, \ldots, A_p\}$, alors $r(S)$ dénote l'ensemble des valeurs prises par le *n-uplet* r pour les attributs appartenant à S.

Toujours par définition, les lignes forment un ensemble, car il n'y a pas de ligne en double et, de la même manière que les attributs, les lignes ne sont pas ordonnées.

Par exemple, $Dom(id) = \mathbb{N}^*$ et l'extension de la relation adhérents est représentée par le tableau 4.3.

3.3. Notion de clés

Pour repérer un *n-uplet* dans l'ensemble des *n-uplets* d'un type donné (c'est-à-dire un adhérent parmi tous les adhérents), il est nécessaire de choisir un attribut qui identifie de manière unique ce *n-uplet*.

Dans l'exemple des adhérents, il n'y a pas de raison que le nom ou le prénom soit unique dans la base de données. En revanche, l'adresse e-mail a plus de chance de l'être.

Définition 4.2. Clés

Un attribut qui permet d'identifier de manière unique un *n-uplet* est nommé **clé**. Un *n-uplet* peut être identifiée par plusieurs clés ; dans ce cas, il convient de définir une **clé primaire** et des **clés secondaires** (utile pour la gestion des algorithmes de recherche non détaillés dans cet ouvrage).

Dans le cas des adhérents, l'adresse e-mail peut donc être une clé primaire (si l'on impose qu'elle soit unique : les membres d'une même famille doivent prendre une adresse e-mail différente).

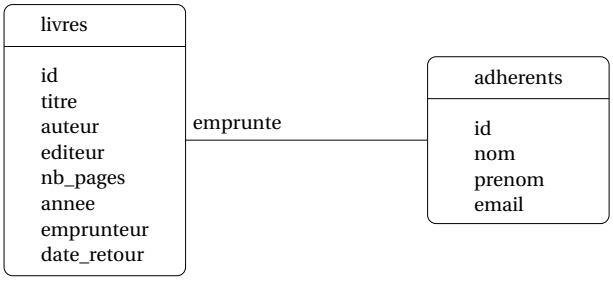
Dans le cas des livres, rien ne garanti l'unicité de chacun de ses attributs : plusieurs livres pourraient avoir le même titre. Il faut donc ajouter un attribut qui permettra d'identifier de

manière unique les livres dans la base de données. En pratique, dans la majorité des cas, la **clé primaire** sera un identifiant abstrait unique de type nombre ; souvent noté **id**.

Toujours en pratique, la clé primaire doit :

- identifier de manière unique un *n-uplet* parmi l'ensemble des *n-uplets* de même type ;
- ne jamais être modifiée (dans ce cas, l'adresse e-mail n'est pas bien adaptée) ;
- prendre un minimum d'espace mémoire pour des raisons d'efficacité des recherches.

La base de données est maintenant représentée par le schéma suivant.



Remarque

Pour définir qui a emprunté un livre, l'attribut *emprunteur* d'un livre fera référence à l'identifiant *id* de l'adhérent.

4. L'utilisation d'un logiciel pour manipuler des bases de données

L'utilisation d'une base de données passe la plupart du temps par une application qui va faire des requêtes à la base de données pour ensuite traiter les résultats. La grande majorité de l'utilisation des bases de données se situe au travers des sites web (avec les langages PHP, AJAX...) ou les applications sur tablette et smartphone.

Pour ne pas avoir à apprendre d'autres langages de programmation, le programme d'informatique propose d'utiliser les bases de données à travers une interface graphique. Nous faisons le choix, dans cet ouvrage, d'utiliser des bases de données SQLite, qui est une implémentation des bases SQL dans un fichier. SQLite ne nécessite pas l'installation d'un serveur SQL. Il existe différentes interfaces graphiques permettant de dialoguer avec des bases de données SQLite. Nous utiliserons ici SQLiteManager, un module complémentaire de Firefox, mais vous pouvez aussi utiliser SQLiteBrowser qui est un logiciel à part entière (`http://sqlitebrowser.org/`).

4.1. Installation de SQLiteManager

Il faut commencer par installer Firefox à partir de l'adresse :
`https://www.mozilla.org/fr/firefox/`
Il suffit ensuite d'ajouter l'extension en allant à l'adresse :
`https://addons.mozilla.org/fr/firefox/addon/sqlite-manager/`

Il faut ensuite aller dans la barre des menus, puis la rubrique outils et cliquer sur SQLiteManager. Dans le cas où la barre de menu est invisible, faire un clic droit sur la barre de l'application en haut et cocher « Barre de menus ».

Une fenêtre identique à celle de la figure 4.3 doit s'ouvrir.

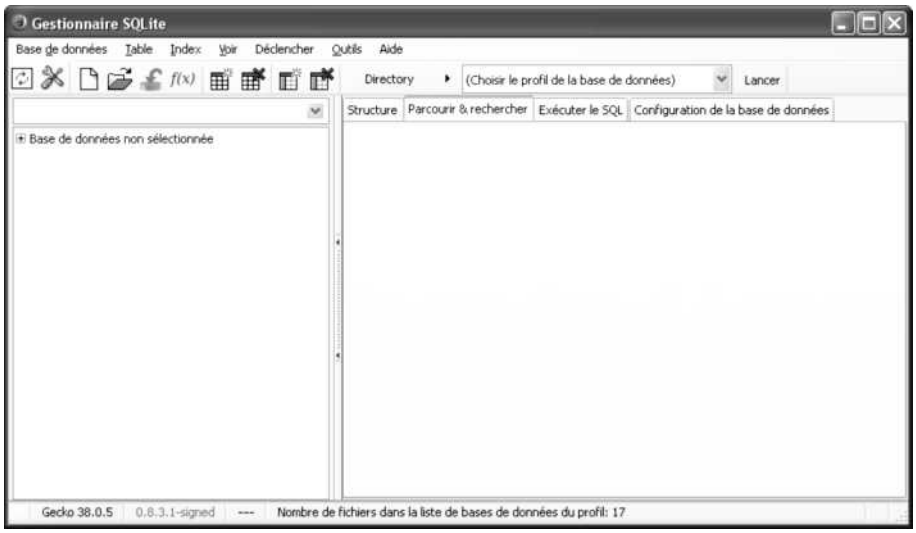


Figure 4.3. Application SQLiteManager.

4.2. Utilisation de SQLiteManager

Dans cette fenêtre, vous pouvez créer une nouvelle base de données, puis ajouter une nouvelle table en renseignant les éléments nécessaires (figure 4.4).

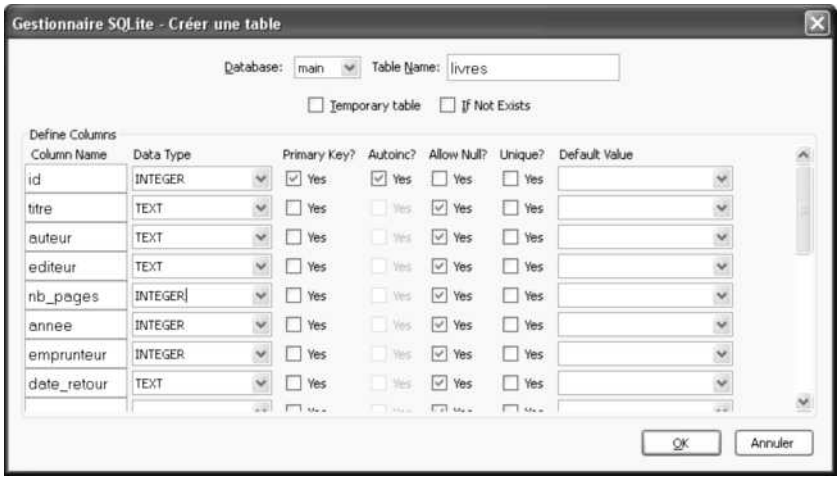


Figure 4.4. Création d'une table graphiquement avec SQLiteManager.

Dans l'onglet « Structure » de la table, il est possible d'ajouter une ou plusieurs colonnes.

Dans l'onglet « Parcourir et rechercher », il est possible de voir les différents éléments de la table, de faire des recherches simples, d'ajouter des lignes.

Dans l'onglet « Exécuter le SQL », il est possible de rentrer des commandes SQL et de les exécuter, comme nous le verrons dans le paragraphe 5.

Pour tester, vous pouvez essayer de créer les tables « livres » et « adherents » de l'exemple de la bibliothèque.

Vous pouvez également prendre la version finale dans le dossier ressource. Pour charger la base de données, cliquer sur « Base de données », « Connecter la base de données », puis sélectionner le fichier `bibliotheque.sqlite`.

5. L'algèbre relationnelle

Une table de la base de données peut être considérée comme un ensemble de *n-uplets* sur lequel on peut appliquer des opérations issues de la théorie des ensembles et au-delà.

Une table est aussi appelée une relation, notée R. Une relation est composée :

- d'éléments : les éléments de la relation seront notés r ;
- d'un schéma : liste des différents attributs avec leur domaine. On notera les schémas S ;
- d'une extension : liste des valeurs de la table.

Dans le cadre du programme de CPGE, il faut être capable de traduire une opération de l'algèbre relationnelle en une requête SQL pouvant s'appliquer sur une base de données classique.

5.1. Projection

La projection consiste à restreindre les éléments de la relation sélectionnée, c'est-à-dire à sélectionner des colonnes de la table.

Définition 4.3. Projection

Soit R une relation composée d'un schéma S. Soit $X \subset S$ une sous-partie du schéma. La projection de la relation R selon X est donnée par :

$$\Pi_X(R) = \{r(X) | r \in R\}$$

où $r(X)$ est la restriction de r sur les attributs de X.

Le résultat d'une projection est une nouvelle relation R' composée du schéma X.

Exemple

On réalise la projection de la relation « livres » selon les attributs (titre, auteur), $\Pi_{\text{titre,auteur}}(\text{livres})$. On obtient :

titre	auteur
Notre-Dame de Paris	Victor Hugo
Les misérables	Victor Hugo
Voyage au centre de la Terre	Jules Verne
Le tour du monde en 80 jours	Jules Verne

La traduction d'une projection en langage SQL se fait avec le mot clé : `SELECT` suivi des attributs, puis du mot clé `FROM` suivi du nom de la relation (la table).

Exemple

Pour réaliser la projection de l'exemple précédent, il faut faire la requête :

```
SELECT titre, auteur
FROM livres
```

Remarque

On peut aussi faire une projection sur tous les attributs en mettant l'opérateur *. La requête `SELECT * FROM livres` renverra la table complète.
Il faut éviter au maximum d'utiliser la sélection complète, car elle est très consommatrice de ressources (bande passante, processeur) et peut entraîner des erreurs au niveau de l'application cliente en cas d'ajouts d'attributs dans la base de données.

5.2. Sélection

Une sélection consiste à rechercher des éléments d'une table (éventuellement le résultat d'une projection) vérifiant un ou plusieurs critères.

Définition 4.4. Sélection

Soit R une relation composée d'un schéma S. La sélection suivant un critère C est donnée par

$$\sigma_C(R) = \{r(S) | r \in R, C(r)\}$$

avec $C(r)$ une condition construite à partir des attributs, de fonctions usuelles, de constantes.

Exemple

La sélection dans « livres » des ouvrages écrits avant 1850, $\sigma_{\text{annee}<1850}$(livres), conduit à :

id	titre	auteur	editeur	nb_pages	annee	emprunteur	date_retour
1	Notre-Dame de Paris	Victor Hugo	Gosselin	636	1831	1	05/06/2014

La sélection en langage SQL s'écrit en ajoutant les critères derrière le mot clé `WHERE` dans une commande classique de projection.

Exemple

Pour réaliser la sélection précédente, il faut écrire la requête :

```
SELECT * FROM livres
WHERE annee < 1850
```

Opérateurs de comparaisons Les opérateurs de comparaison classiques sont les suivants :

opérateur	désignation
<	inférieur strictement
>	supérieur strictement
<=	inférieur ou égal
>=	supérieur ou égal
!= ou <>	différent
=	égal
IS NULL	est nul
IS NOT NULL	est non nul

La valeur NULL correspond à l'élément vide. Par conséquent, le test « `a IS NULL` » est différent de « `a != 0` ».

Opérateurs OR ou AND Pour réaliser des sélections plus complètes, plusieurs critères peuvent être associés avec OR ou AND.

Exemple

La requête suivante permet de sélectionner les livres parus après 1850 et contenant plus de 1 000 pages, $\sigma_{\text{annee>1850 and nb_pages>1000}}(\text{livres})$:

```
SELECT *
FROM livres
WHERE ( annee > 1850 AND nb_pages > 1000)
```

Elle peut être vue comme l'intersection de deux ensembles :
$\sigma_{\text{annee>1850}}(\text{livres}) \cap \sigma_{\text{nb_pages>1000}}(\text{livres})$.

Exemple

La requête suivante permet de sélectionner les livres dont l'éditeur est Lacroix ou Hetzel, $\sigma_{\text{editeur= "Lacroix" ou editeur = "Hetzel"}}(\text{livres})$:

```
SELECT *
FROM livres
WHERE ( editeur = "Lacroix" OR editeur = "Hetzel")
```

Elle peut être vue comme l'union de deux ensembles :
$\sigma_{\text{editeur= "Lacroix"}}(\text{livres}) \cup \sigma_{\text{editeur= "Hetzel"}}(\text{livres})$

Remarque

Il ne faut pas oublier que le AND est prioritaire sur le OR, en mettant des parenthèses si nécessaire.

Opérateurs IN Plutôt que de faire une très longue suite de OR, il est préférable de rechercher si l'élément est dans une liste avec l'opérateur IN suivi de la liste.

Exemple

Reprenons l'exemple précédent de la sélection des livres dont l'éditeur est Lacroix ou Hetzel. La requête suivante est un équivalent plus concis, $\sigma_{\text{editeur}\in\{\text{"Lacroix","Hetzel"}\}}(\text{livres})$:

```
SELECT *
FROM livres
WHERE ( editeur IN ("Lacroix", "Hetzel"))
```

Opérateurs BETWEEN Cet opérateur permet de chercher un critère ordonné entre deux valeurs : `BETWEEN valeur1 AND valeur2`.

Exemple

La requête suivante permet de sélectionner les livres dont le nombre de pages est compris entre 300 et 1 000, $\sigma_{\text{nb_pages}>=300 \text{ and } \text{nb_pages} <=1000}(\text{livres})$:

```
SELECT *
FROM livres
WHERE ( nb_pages BETWEEN 300 AND 1000)
```

Remarque

La recherche peut se faire entre deux valeurs numériques, deux dates et même deux chaînes de caractères.

Opérateurs LIKE Cet opérateur permet de faire une recherche sur un modèle plutôt que sur une valeur exacte.

Exemple

La requête suivante permet de sélectionner les livres dont les titres contiennent la chaîne `"en"` :

```
SELECT *
FROM livres
WHERE ( titre LIKE "%en%" )
```

De même, on peut sélectionner :

- les mots commençant par a avec `LIKE "a%"` ;
- les mots finissant par a avec `LIKE "%a"` ;
- les commançant par a et finissant par b avec `LIKE "a%b"`.

Les chaînes de caractères comportant le symbole % sont des « expressions régulières ».

Opérateurs DISTINCT Il peut également être intéressant de n'afficher que les résultats distincts les uns des autres. Par exemple, la requête suivante permet de sélectionner les auteurs de la table livres de manière unique :

```
SELECT DISTINCT(auteur)
FROM livres
```

Cette requête renvoie le tableau suivant

auteur
Jules Verne
Victor Hugo

Synthèse

Initiation aux bases de données

Les bases de données ont pour objectif de stocker un grand nombre de données afin de pouvoir être utilisées par des applications externes, dites *clientes*.

Architecture L'architecture matérielle utilisée pour les applications dépend de celles-ci, et notamment du nombre de clients qui devront accéder aux données.

Si l'accès n'est fait que par un seul client, comme un logiciel de messagerie, la base de données est située sur la même machine que le client : il s'agit d'une architecture à 1 niveau.

Si l'accès aux données doit être fait par des milliers de clients simultanément, il faut utiliser des architectures à N niveaux (avec N au moins égal à 3) sur lesquelles la charge de travail est répartie pour réduire au maximum les temps d'attente des clients lors des requêtes. C'est le cas pour les architectures des serveurs internet notamment.

Modélisation des bases de données Une base de données est un ensemble structuré d'informations qui permet de répondre à un besoin spécifique.

Un des objectifs est de limiter la redondance des informations en modélisant la réalité par des entités et des associations entre ces entités. Dans le cadre d'une bibliothèque, il y aura des entités pour modéliser les livres, les emprunteurs, les auteurs... Puis il sera créé des associations permettant de définir comment interagissent les entités entre elles.

Un ensemble d'entités de même type sera représenté simplement par un tableau à deux dimensions. Les colonnes du tableau sont les *attributs* prenant des valeurs dans un certain domaine. Chaque ligne du tableau est un *n-uplet* ou une *entité*. Chaque entité étant unique, l'ensemble des entités est un ensemble au sens mathématique.

Il est possible de restreindre l'ensemble des entités à un sous-ensemble de ses attributs. Le résultat sera à nouveau un ensemble. Il est à noter qu'il n'y a pas de notions d'ordre dans ces ensembles.

Pour différencier les entités entre elles, on définit une *clé primaire* pour chaque ensemble d'entités. Cette clé primaire, souvent un entier, permet d'identifier de manière unique chaque entité dans son ensemble.

Algèbre relationnelle et requête SQL Dans le cadre du programme de CPGE, il faut être capable de créer des requêtes au format SQL permettant de réaliser un certain nombre d'opérations basiques. Comme l'objectif n'est pas de concevoir une application cliente utilisant une base de données, les requêtes seront directement utilisées dans un gestionnaire graphique de base de données comme SQLiteManager, pour les bases de données SQLite, ou PhpMyAdmin, pour les bases de données MySQL.

Soit une base de données composée des tables `table1` et `table2`. La table `table1` est constituée des attributs A1, B1, C1 et la table `table2` des attributs A2, B2 et C2.

Projection	Restriction de l'ensemble d'origine sur les attributs demandés.	`SELECT A1, B1 FROM table1`

Sélection	Sélection d'un sous-ensemble des entités d'une table donnée suivant un critère.	`SELECT * FROM table1`
		`WHERE (A1 LIKE "critere")`
		`SELECT A1,B1 FROM table1`
		`WHERE (A1 LIKE "critere")`

Opérateurs de comparaison		<, >, <=, >=, =!, <>, =
	L'élément `NULL` représente l'ensemble vide et il est possible de tester si un élément contient l'ensemble vide ou non.	`IS NULL` `IS NOT NULL`
	Test d'un élément dans une liste.	`A1 IN (elem1, elem2, elem3)`
	Recherche des éléments entre deux valeurs.	`A1 BETWEEN val1 AND val2`
	Recherche sur un modèle pour trouver les éléments dont la fin de la chaîne de caractère est `"abc"`.	`A1 LIKE "%abc"`
	Combinaison des critères.	`AND, OR`
	Sélection des éléments distincts.	`SELECT DISTINCT(A1) FROM table1`

Opérateurs ensemblistes	Union de deux ensembles ?	`SELECT * FROM table1`
		`UNION`
		`SELECT * FROM table2`

	Intersection de deux ensembles.	`SELECT * FROM table1`
		`INTERSECT`
		`SELECT * FROM table2`

	Différence de deux ensembles.	`SELECT * FROM table1`
		`EXCEPT`
		`SELECT * FROM table2`

Produit cartésien	Produit cartésien de deux ensembles : fusion des deux ensembles en créant toutes les combinaisons possibles entre les entités des deux ensembles.	`SELECT * FROM table1, table2`
Jointure	Jointure de deux tables : équivalent à un produit cartésien en suivant un critère de fusion.	`SELECT * FROM table1` `JOIN table2 ON A1=B2`
Opérateurs d'agrégat	Minimum d'un attribut.	`SELECT MIN(A1) FROM table1`
	Maximum d'un attribut.	`SELECT MAX(A1) FROM table1`
	Moyenne d'un attribut.	`SELECT AVG(A1) FROM table1`
	Somme des attributs.	`SELECT SUM(A1) FROM table1`
Organisation des résultats	ordonnancement par ordre croissant	`SELECT A1 FROM table1 ORDER BY A1 ASC`
	Ordonnancement par ordre décroissant.	`SELECT A1 FROM table1 ORDER BY A1 DESC`
	Récupération N1 valeurs à partir du N-ième résultat.	`SELECT A1 FROM table1 LIMIT N, N1`

Exercices

Initiation aux bases de données

Vrai ou faux ?

En reprenant la définition des deux tables de la bibliothèque définie dans le cours (tables 4.4 et 4.5), répondre aux questions suivantes.

		Vrai	Faux
a)	La requête SQL : `SELECT titre FROM livres ORDER BY titre ASC` renvoie la liste : `["Notre-Dame de Paris", "Les misérables", "Voyage au centre de la Terre", Le tour du monde en 80 jours"]`.	☐	☐
b)	La requête SQL : `SELECT count(nb_pages), sum(nb_pages) FROM livres2` renvoie les entiers 3 et 1 613.	☐	☐
c)	La requête SQL : `SELECT * FROM (SELECT * from livres UNION SELECT * from livres2) WHERE (emprunteur = "" AND titre = "Les misérables")` permet de savoir si le livre *Les misérables* est disponible et dans quelle bibliothèque il se trouve.	☐	☐
d)	La requête SQL : `SELECT DISTINCT titre FROM (SELECT * FROM livres UNION SELECT * FROM livres2) WHERE auteur = "Victor Hugo"` renvoie la liste `["Notre-Dame de Paris", "Les misérables"]`.	☐	☐

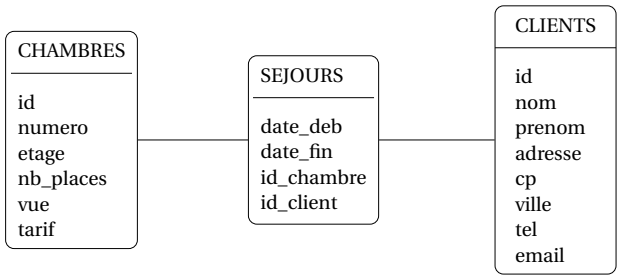
Exercices

Certains exercices utilisent des fichiers disponibles dans un dossier ressource en libre téléchargement à la page du livre sur le site `http://www.vuibert.fr/`.

●○○ Exercice 1 Base de données pour la gestion des chambres d'un hôtel (25 min.)

Le service de réservation d'un hôtel utilise un logiciel qui accède à une base de données qui contient toutes les informations sur les chambres, les clients, les séjours...

Dans sa version la plus élémentaire, la base de données de cet hôtel est constituée de trois entités. Elle est résumée par le schéma ci-dessous.



L'entité CHAMBRES contient les données sur les chambres de l'hôtel avec les attributs :

- id : identifiant, de type entier, clé primaire ;
- numero : numéro de la chambre, de type entier ;
- etage : numéro de l'étage où se situe la chambre, de type entier ;
- nb_places : nombre de couchage dans la chambre, de type entier ;
- vue : précise si la vue de la chambre est « parking » ou « route », de type texte ;
- tarif : tarif de la chambre pour une nuitée avec petit-déjeuner inclus.

L'entité SEJOURS contient la liste des réservations et séjours avec les attributs :

- date_deb : date de début du séjour, de type date, au format 2015/01/25 ;
- date_fin : date de fin du séjour de type date, au format 2015/01/25 ;
- id_chambre : identifiant de la chambre réservée, de type entier ;
- id_client : identifiant du client, de type entier.

L'entité CLIENTS contient la liste des clients avec les attributs :

- id : identifiant, de type entier, clé primaire ;
- nom : nom du client, de type texte ;
- prenom : prénom du client, de type texte ;
- nom : nom du client, de type texte ;
- adresse : adresse du client, de type texte ;
- cp : code postal du client, de type entier ;
- ville : ville du client, de type texte ;
- tel : numéro de téléphone du client, de type texte ;
- email : adresse e-mail du client, de type texte.

1) Monsieur MARTIN Jean arrive le 10 avril 2015 à l'accueil de l'hôtel et souhaite prendre possession de sa chambre. Donner la requête SQL permettant de connaître le numéro et l'étage de la chambre qui lui a été attribuée.

2) Monsieur MARTIN Jean souhaiterait connaître le nombre de séjours qu'il a fait dans cet hôtel : donner la requête SQL.
3) Monsieur MARTIN Jean souhaiterait savoir plus précisément le nombre de séjours effectués en 2015 : donner la requête SQL.
4) Une famille de quatre personnes appelle pour réserver une chambre du mercredi 13 mai 2015 au dimanche 17 mai 2015. Donner la requête SQL permettant de connaître le numéro et l'étage des chambres libres.
5) Plus précisément, la famille souhaiterait avoir la vue sur le parking. Donner la requête SQL permettant de connaître le numéro et l'étage des chambres libres.

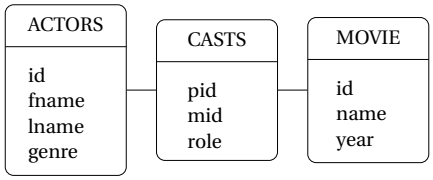
●●○ Exercice 2 Recherche de films dans une base de données (40 min.)

Il existe beaucoup de sites permettant de lister les productions cinématographiques mondiales, les acteurs, les producteurs, etc.

Le site `http://www.imdb.com/` est l'un des plus populaires. Cet exercice va se baser sur une partie de la base de données de ce site afin de réaliser des requêtes permettant de rechercher des informations.

Dans sa version la plus élémentaire, la base de données permettant de gérer les films et les acteurs est composée de deux entités différentes, une pour les acteurs et une autre pour les films. Afin de stocker l'information « tel acteur joue dans tel film en jouant tel rôle », il est nécessaire de créer une entité supplémentaire appelée castings.

La solution est résumée par le diagramme ci-contre.



L'entité ACTORS contient les acteurs avec les attributs :

- id : identifiant de type entier, clé primaire ;
- fname : prénom, de type texte ;
- lname : nom, de type texte ;
- genre : le genre, de type texte.

L'entité CASTS contient les castings avec les attributs :

- pid : identifiant d'un acteur, de type entier ;
- mid : identifiant d'un film, de type entier ;
- role : rôle tenu par l'acteur (pid) dans le film (mid) (soit le nom du personnage), de type texte.

L'entité MOVIE contient les films avec les attributs :

- id : identifiant, de type entier, clé primaire ;
- name : titre du film, de type texte ;
- year : année du film, de type entier.

Décompresser l'archive imdb2010.sqlite.zip, disponible dans le dossier ressource, (bouton droit sur le fichier, puis décompresser sur la plupart des systèmes d'exploitation). Puis ouvrir la base de données à l'aide de SQLite Manager ou tout autre gestionnaire de base de données SQLite.

1) Utiliser maintenant l'application graphique pour :
   - modifier des données ;
   - faire des recherches simples.

Pour réaliser des recherches « complexes », il faut passer par les commandes SQL.

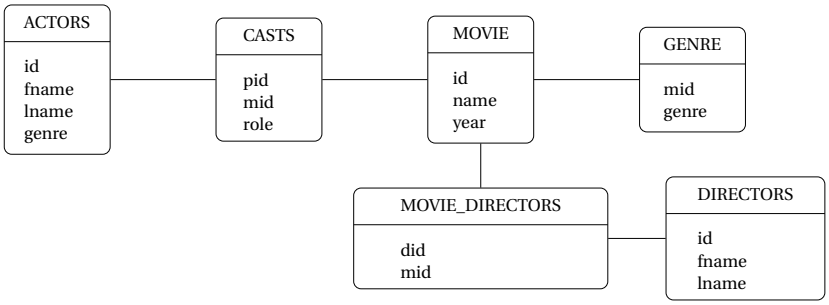
> **Remarque**
> Vous pouvez aller sur le site `http://sql.sh/cours` qui recence toutes les commandes SQL en détaillant la syntaxe et en donnant des exemples.
>
> Pour construire une commande SQL complexe, il faut l'écrire étape par étape et vérifier que chaque étape renvoie le résultat escompté.

2) Déterminer la requête SQL permettant de récupérer toutes les informations des acteurs dont le nom est « Fox ». Mots clés : `SELECT, WHERE, LIKE`.
3) Déterminer la requête SQL permettant de récupérer les prénoms et noms des acteurs dont le nom est « Fox ». Mots clés : `SELECT, WHERE, LIKE`.
4) Déterminer la requête SQL permettant de récupérer l'identifiant des films dans lesquels l'acteur « James Avery » joue. Mots clés : deux commandes `SELECT` imbriquées.
5) Déterminer la requête SQL permettant de récupérer les titres et années des films dans lesquels l'acteur « James Avery » joue. Mots clés : trois commandes `SELECT` imbriquées, utiliser `IN`.
6) Déterminer la requête SQL permettant de récupérer les identifiants des films dans lesquels jouent les acteurs « James Avery » et « Shia LaBeouf » ensemble. Mots clés : commande `SELECT` et `INTERSECT`.
7) Modifier la réquête précédente pour obtenir les titres et années des films dans lesquels jouent les acteurs « James Avery" et « Shia LaBeouf » ensemble.
8) Déterminer la requête SQL permettant de récupérer le nombre d'acteurs (référencés) jouant dans le film *Transformers 3*. Mots clés : `COUNT`.
9) Déterminer la requête SQL permettant de récupérer le casting : prénom, nom des acteurs et leur rôle dans le film *Transformers 3*. Mots clés : `table1 JOIN table2 ON`...

●●● Exercice 3 Recherche dans une base de données de films – Requêtes avancées (50 min.)

La base de données du site IMDB contient en réalité beaucoup plus de données. On rajoute quelques tables dans cette partie afin d'aller explorer des requêtes plus complexes dans le but d'effectuer des recherches sur plus de champs en même temps.

La base de données étudiée dans cet exercice possède la même structure que précédemment avec quelques tables en plus.



L'entité GENRE contient les genres des films avec les attributs :

- mid : identifiant d'un film, de type entier ;
- genre : genre du film, de type texte.

L'entité MOVIE_DIRECTORS contient les liens entre les metteurs en scène et les films avec les attributs :

- did : identifiant d'un metteur en scène, de type entier ;
- mid : identifiant d'un film, de type entier.

L'entité DIRECTORS contient les metteurs en scène avec les attributs :

- id : identifiant, de type entier, clé primaire ;
- fname : prénom du metteur en scène, de type texte ;
- lname : nom du metteur en scène, de type texte.

1) Lister les prénoms et noms de tous les acteurs jouant dans le film *Bienvenue chez les Ch'tis*. Vous devez obtenir 29 acteurs...
2) Déterminer les metteurs en scène des films du genre « War » tournés une année multiple de 4. La requête devra retourner les prénoms, noms des metteurs en scène, titre du film et année. *Aide : Utiliser l'opérateur modulo % dans la requête ainsi que deux jointures. Travailler étape par étape !*
3) Déterminer les metteurs en scène qui ont tourné plus de 500 films. La requête retournera leurs prénom, nom et nombre de films par ordre décroissant. *Aide : Utiliser count, having, group by, order by.*
4) Déterminer les acteurs qui ont joué 5 rôles ou plus dans le même film au cours de l'année 2010. Attention, la base de données peut contenir des erreurs et un acteur peut être rentré plusieurs fois avec le même rôle pour un film. Ici, on souhaite avoir des rôles distincts. La requête doit retourner le prénom et le nom des acteurs, le nom du film, le nombre de rôles distincts joués dans le film.

Corrigés

Initiation aux bases de données

Corrigés des Vrai/Faux

a) Faux, la commande `ORDER BY titre ASC` renvoie la liste triée par ordre alphabétique : `["Le tour du monde en 80 jours", "Les misérables", "Notre-Dame de Paris", "Voyage au centre de la Terre"]`.

b) Vrai.

c) Faux. La requête permet de savoir si le livre est disponible en renvoyant le nombre d'exemplaires disponibles, mais elle ne permet pas de connaître la bibliothèque dans laquelle l'ouvrage sera disponible.

d) Vrai.

Corrigés des exercices

Exercice 1

1)
```
SELECT numero, etage FROM CHAMBRES WHERE id =
SELECT id_chambre FROM SEJOURS WHERE
(id_client = SELECT id FROM CLIENTS WHERE nom = "MARTIN" and prenom = "Jean") and
(date_deb = "2015-04-10")
```

2)
```
SELECT count(id_client) FROM SEJOURS WHERE
(id_client = SELECT id FROM CLIENTS WHERE nom = "MARTIN" and prenom = "Jean")
```

3)
```
SELECT count(id_client) FROM SEJOURS WHERE
(id_client = SELECT id FROM CLIENTS WHERE nom = "MARTIN" and prenom = "Jean")
and date_deb >= "2015-01-01"
and date_fin <= "2015-12-31"
```

4)
```
SELECT numero, etage FROM CHAMBRES WHERE nb_places >=4 AND id NOT IN (
SELECT id_chambre FROM SEJOURS WHERE
date_deb between '2015/05/13' and '2015/05/17' or
date_fin between '2015/05/13' and '2015/05/17')
```

5)
```
SELECT numero, etage FROM CHAMBRES WHERE id NOT IN (
SELECT id_chambre FROM SEJOURS WHERE
date_deb between '2015/05/13' and '2015/05/17' or
date_fin between '2015/05/13' and '2015/05/17')
AND vue = "parking"  AND nb_places >=4
```

Exercice 2

1) Pas de réponse attendue.

2)
```
SELECT * FROM ACTORS WHERE lname LIKE "Fox"
```

3)
```
SELECT fname,lname FROM ACTORS WHERE lname LIKE "Fox"
```

4)
```
SELECT mid FROM CASTS
WHERE pid = (SELECT id FROM ACTORS WHERE lname LIKE "Avery" and fname="James")
```

5)
```
SELECT name, year FROM MOVIE
WHERE id in (SELECT mid FROM CASTS
WHERE pid = (SELECT id FROM ACTORS WHERE lname LIKE "Avery" and fname="James"))
```

6)
```
SELECT mid FROM CASTS WHERE pid = (SELECT id FROM ACTORS
WHERE fname LIKE "Shia" and lname LIKE "LaBeouf")
INTERSECT SELECT mid FROM CASTS WHERE
pid = (SELECT id FROM ACTORS WHERE fname LIKE "James" and lname LIKE "Avery")
```

7)
```
SELECT name, year FROM MOVIE WHERE id in
(SELECT mid FROM CASTS
WHERE pid = (SELECT id FROM ACTORS WHERE fname LIKE "Shia" and lname LIKE "LaBeouf")
INTERSECT SELECT mid FROM CASTS
WHERE pid = (SELECT id FROM ACTORS WHERE fname LIKE "James" and lname LIKE "Avery"))
```

8)
```
SELECT COUNT(pid) FROM CASTS
WHERE mid = (SELECT id FROM MOVIE WHERE name LIKE "Transformers 3")
```

9)
```
SELECT fname, lname, role FROM CASTS JOIN ACTORS ON pid=id
WHERE mid = (SELECT id FROM MOVIE WHERE name LIKE "Transformers 3")
```

Exercice 3

1)
```
SELECT fname, lname FROM ACTORS WHERE id in (
SELECT pid FROM CASTS WHERE mid = (
SELECT id FROM MOVIE WHERE name LIKE "Bienvenue chez les Ch'tis"))
```

2)
```
SELECT fname,lname,name,year
from movie_directors INNER directors ON directors.id=did
JOIN MOVIE on mid=movie.id where mid in (
SELECT id from MOVIE WHERE id in (
SELECT mid FROM GENRE WHERE genre = "War") and year %4 =0)
```

3)
```
SELECT fname, lname, count(mid)
FROM movie_directors JOIN directors ON id=did
GROUP BY did having count(mid) > 500
ORDER BY count(mid) desc
```

4)
```
SELECT fname,lname,name, count(distinct role)
FROM casts JOIN actors ON pid=actors.id JOIN movie ON mid=movie.id
WHERE year = 2010
GROUP BY mid, pid
HAVING count(distinct role) > 4
```

Chapitre 6

Problèmes de synthèse

Certains problèmes utilisent des fichiers disponibles dans un dossier ressource en libre téléchargement à la page du livre sur le site `http://www.vuibert.fr/`.

1. Les checksums et la correction des erreurs [1]

1.1. Sujet



Les données échangées entre systèmes numériques au travers des différents supports physiques (câbles électriques, fibres optiques, ondes électromagnétiques, etc.) peuvent être perturbées par toutes sortes de facteurs pouvant provoquer des erreurs dans les données : autres signaux électromagnétiques, masses métalliques, imperfections du matériel électronique, etc. En pratique, il est donc indispensable de pouvoir détecter ces erreurs et, dans la mesure du possible, de les corriger sans que cela ne nécessite une nouvelle transmission. L'objet de cette partie est de mettre en place quelques algorithmes dans ce but.

1.1.1. Bit de parité

Une technique, simple et très répandue, pour s'assurer qu'une donnée transmise sous la forme d'un mot binaire sera lue correctement par son récepteur est de lui adjoindre un bit de parité, égal par définition à :

- 0, si la donnée contient un nombre pair de 1 (et donc si ses bits sont de somme paire) ;
- 1, si la donnée contient un nombre impair de 1 (et donc si ses bits sont de somme impaire).

Après réception de la donnée, le récepteur recalcule le bit de parité et le compare à celui que l'émetteur lui a adressé. Si la donnée n'a pas été altérée lors de la transmission, alors les deux bits de parité sont forcément identiques.

1) Donner les bits de parité associés aux représentations binaires des entiers 5, 76 et 37.
2) Écrire une fonction `parite(bits)` prenant pour argument une liste de bits constituée d'entiers valant 0 ou 1 et retournant l'entier 0 ou 1 correspondant à son bit de parité.

[1] Adapté d'un sujet de la banque PT.

Les techniques de vérification les plus simples consistent à découper la donnée en blocs et à joindre un bit de parité à chaque bloc. Par exemple, certains protocoles (comme la liaison série UART) transmettent sept ou huit bits de données pour un bit de parité.

3) Donner un exemple de mauvaise transmission qui n'est pas détectable par cette technique. Si une erreur a été détectée, est-il possible de la corriger sans retransmettre la donnée ?

1.1.2. Code de Hamming

Le code de Hamming est un exemple d'utilisation des bits de parité pour détecter et corriger des erreurs. Nous nous intéressons ici au code dit (7,4), ainsi appelé parce qu'il consiste à joindre trois bits de parité à quatre bits de données, ce qui donne un message d'une longueur totale de sept bits. Ces trois bits de parité sont définis ainsi : si la donnée s'écrit (d_1, d_2, d_3, d_4) avec $d_1 = 0$ ou 1, alors :

- p_1 est le bit de parité du triplet (d_1, d_2, d_4) ;
- p_2 est le bit de parité du triplet (d_1, d_3, d_4) ;
- p_3 est le bit de parité du triplet (d_2, d_3, d_4).

Le message encodé, que l'on transmet, s'écrit alors comme suit : $(p_1, p_2, d_1, p_3, d_2, d_3, d_4)$.

4) Écrire une fonction `encode_hamming(donnee)` prenant pour argument une liste donnée de quatre bits (représentés par des entiers valant 0 ou 1) et retournant une liste de bits contenant le message encodé. On pourra appeler la fonction `parite(bits)` précédemment définie.

Le contrôle après réception d'un message ainsi encodé est relativement simple. On pourrait naturellement recalculer les trois bits de parité de la donnée et les comparer aux valeurs transmises, mais la technique proposée par Hamming est de calculer les trois bits de contrôle suivants, notés (c_1, c_2, c_3), à partir du message complet (données et bits supplémentaires), noté $(m_1, \ldots, m_7)$:

- c_1 est le bit de parité de l'ensemble (m_4, m_5, m_6, m_7) ;
- c_2 est le bit de parité de l'ensemble (m_2, m_3, m_6, m_7) ;
- c_3 est le bit de parité de l'ensemble (m_1, m_3, m_5, m_7).

On montre que, si le message a bien été encodé selon les règles précédentes et n'a pas été altéré, alors les trois bits de contrôle doivent être à 0. Si ce n'est pas le cas, alors il y a eu une erreur ; l'intérêt de la technique de Hamming est que, dans le cas particulier où l'erreur est unique, le mot de contrôle donne la représentation binaire de la position de cette erreur en numérotant à partir de 1. Par exemple, si $(c_1, c_2, c_3) = (0, 1, 1)$, alors l'erreur porte sur le troisième bit du message. Il suffit ainsi d'inverser ce bit (le mettre à 1 s'il est à 0, et inversement) pour corriger l'erreur.

La donnée décodée est alors constituée des quatre bits (d_1, d_2, d_3, d_4) qui se trouvent respectivement en positions 3, 5, 6 et 7 (toujours en numérotant à partir de 1), conformément à la description de l'encodage donnée ci-dessus.

5) Écrire une fonction `decode_hamming(message)` prenant pour argument une liste de sept bits et retournant une liste de quatre bits contenant la donnée décodée. En cas d'erreur, on affichera à l'écran un avertissement indiquant la position du bit affecté et on effectuera la correction. On supposera dans cette question que, s'il y a une erreur, alors elle est unique.
6) Déterminer le codage de Hamming de la donnée `1011`, puis la donnée décodée par l'algorithme dans l'hypothèse où les deux premiers bits du message codé ont été incorrectement transmis. Quel a été l'effet de la « correction » sur la donnée dans ce cas ?

7) Sans coder, proposer un moyen simple de différencier une double erreur d'une erreur unique au moyen d'un bit de parité supplémentaire et expliquer comment cela permet d'éviter le problème mis en évidence à la question précédente. On s'appuiera sur les techniques introduites dans cette partie. On ne demande pas d'essayer de corriger la double erreur.

1.2. Correction

1.2.1. Bit de parité

1) L'entier 5 s'écrit en binaire $(101)_2$ et contient un nombre pair de 1. Le bit de parité est donc de 0.
   De même, 16 s'écrit en binaire $(10000)_2$, soit une parité de 1, et 37 s'écrit en binaire $(100101)_2$ soit une parité de 1.
2) Le contrôle de parité consiste à compter le nombre de 1 dans le mot binaire. La parité est obtenue en calculant le reste de la division par deux (la somme modulo 2).

Python

```
def parite(bits):
    somme=0
    for i in range(len(bits)):
        somme=somme+bits[i]
    return somme%2

bits=[1,0,0,1,0,1]   # 37
print(parite(bits))
```

Scilab

```
function p=parite(bits)
    somme=0
    for i =1:size(bits,2)
        somme=somme+bits(i)
    end
    p = modulo(somme,2)
endfunction

bits=[1,0,0,1,0,1]   // 37
disp(parite(bits))
```

3) Le contrôle simple de la parité ne peut pas détecter une double erreur, qui va nécessairement se compenser en terme de parité. Néanmoins, les systèmes de transmission de données sont généralement conçus pour assurer moins d'une erreur tous les 10^5 à 10^7 bits transmis. Il est donc très peu probable de commettre deux erreurs successives.
   Si une erreur est détectée, il est évidemment impossible de la retrouver parmi le mot binaire et de la corriger. Il faut donc demander une retransmission de l'information.

1.2.2. Code de Hamming

4) La fonction doit calculer les trois parités (en faisant attention au décalage d'indice en Python) et construire le mot binaire complet :

Python

```
def encode_hamming(donnee):
    p1=parite([donnee[0],donnee[1],donnee[3]])
    p2=parite([donnee[0],donnee[2],donnee[3]])
    p3=parite([donnee[1],donnee[2],donnee[3]])
    return [p1,p2,donnee[0],p3,donnee[1],donnee[2],donnee[3]]

print(encode_hamming([1,0,1,1]))
```

Scilab

```
function message=encode_hamming(donnee)
    p1=parite([donnee(1),donnee(2),donnee(4)])
    p2=parite([donnee(1),donnee(3),donnee(4)])
    p3=parite([donnee(2),donnee(3),donnee(4)])
    message = [p1,p2,donnee(1),p3,donnee(2),...
                    donnee(3),donnee(4)]
endfunction

disp(encode_hamming([1,0,1,1]))
```

5) L'algorithme doit calculer les parités c_i et calculer la position de l'éventuelle erreur. En cas d'erreur, il faut prévoir la correction (l'inversion du bon bit), puis transmettre les bits de données.

Python

```
def decode_hamming(message):
    c1=parite([message[3],message[4],message[5],message[6]])
    c2=parite([message[1],message[2],message[5],message[6]])
    c3=parite([message[0],message[2],message[4],message[6]])
    pos_erreur=c1*4+c2*2+c3
    if pos_erreur != 0:
        message[pos_erreur-1]=(message[pos_erreur-1]+1)%2
        print("Erreur sur le bit {}".format(pos_erreur))
    return [message[2],message[4],message[5],message[6]]
```

Scilab

```
function donnee=decode_hamming(message)
    c1=parite([message(4),message(5),message(6),message(7)])
    c2=parite([message(2),message(3),message(6),message(7)])
    c3=parite([message(1),message(3),message(5),message(7)])
    pos_erreur=c1*4+c2*2+c3
    if pos_erreur ~= 0
        message(pos_erreur)=modulo(message(pos_erreur)+1,2)
        mprintf("Erreur sur le bit %d",pos_erreur)
    end
    donnee = [message(3),message(5),message(6),message(7)]
endfunction
```

VUIBERT PRÉPAS

INFORMATIQUE POUR TOUS

Classes préparatoires scientifiques
1re et 2e années

Des ouvrages pour faire la différence :
- des cours complets pour acquérir les connaissances indispensables
- des fiches de synthèse pour réviser l'essentiel et acquérir les bons réflexes
- de nombreux exercices intégralement corrigés pour s'entraîner : Vrai/faux, exercices d'application et problèmes de synthèse
- des sujets de concours corrigés pour se mettre en situation d'épreuve
- en ligne (www.vuibert.fr, à la page du livre) : des ressources complémentaires à télécharger

SOMMAIRE

Chapitre 1 : Architecture des ordinateurs
1. Les composants et leurs connexions – 2. L'architecture logicielle – 3. Le codage des données en mémoire

Chapitre 2 : Programmation et algorithmique
1. L'installation et l'utilisation de Python et Scilab – 2. Les structures de données – 3. Les instructions de contrôle – 4. Les fonctions – 5. Le débogage d'un programme – 6. La lecture et l'écriture de fichiers – 7. Les tracés graphiques – 8. La terminaison, la correction et la complexité

Chapitre 3 : Simulation numérique
1. La vectorisation des calculs – 2. La résolution des équations non linéaires – 3. L'intégration et la dérivation de signaux numériques – 4. L'intégration des équations différentielles – 5. La résolution d'un système linéaire par la méthode de Gauss

Chapitre 4 : Initiation aux bases de données
1. Les motivations – 2. L'architecture matérielle et logicielle des bases de données – 3. La modélisation d'une base de données – 4. L'utilisation d'un logiciel pour manipuler des bases de données – 5. L'algèbre relationnelle

Chapitre 5 : Algorithmique : approfondissement
1. Les piles – 2. Les fonctions récursives – 3. Les algorithmes de tri – 4. La programmation orientée objet

Chapitre 6 : Problèmes de synthèse

Les auteurs :

Alain Caignot *est professeur en classes préparatoires scientifiques au Collège Stanislas à Paris*
Marc Dérumaux *est professeur en classes préparatoires scientifiques au lycée Saint-Louis à Paris*
Johann Labasque *est professeur en classes préparatoires scientifiques au lycée Faidherbe à Lille*
Lionel Moisan *est professeur des universités à l'université Paris Descartes*

ISBN : 978-2-311-40099-1



www.Vuibert.fr